

1. (b) Data and a pointer to the next node
2. (c) NULL
3. (b) self-referential structure
4. (b) head == NULL
5. (b) new node $\rightarrow$ next = head; head = new node;
6. (b) temp = head; head = head $\rightarrow$ next; free (temp);
7. (c) traversing from tail to head
8. (b) n

9. (c) copying the data of P $\rightarrow$ next into P, then deleting P $\rightarrow$ next
10. (b) while (P != NULL)
11. (b) P $\rightarrow$ next == NULL

12. (c) Two pointers at different rates

13. (c) Three

14. (b) Exactly one node

15. (b) P = (struct node*) malloc(sizeof(struct node));

16. (b) Avoid special-case handling for insertion and deletion at the end.

17. (c) The first node

18. (b) It does not support random access, (indexed)

19. (b) Loss of access to all nodes, causing a memory leak

20. (b) A slow pointer and a fast pointer

21. (c) The last node

22. (b) Nodes may be scattered anywhere.

23.

struct node

{

int data;

struct node *next;

};

Aryan Dey

UG104/BTCE/2025/031

24.

Given list :

10 → 20 → 30 → 40 → NULL

The program adds each node with the next node:

$$10 + 20 = 30$$

$$20 + 30 = 50$$

$$30 + 40 = 70$$

Aryan Dey

UG104/BTCE/2025/031

Output: 30 50 70

25. void insertAtEnd (struct node **head, int x)

{

struct node *t;

t = (struct node *) malloc (sizeof (struct node));

t->data = x;

t->next = NULL;

if (*head == NULL)

{

*head = t;

return;

}

struct node *p = *head;

while (p->next != NULL)

{

p = p->next;

}

p->next = t;

}

Aryan Dey

UG104/BTCE/2025/031

26. `struct node* reverse (struct node* head)`
`{`
`struct node* prev = NULL;`
`struct node* current = head;`
`struct node* next;`
`while (current != NULL)`
`{`
`next = current->next;`
`current->next = prev;`
`prev = current;`
`current = next;`
`}`
`return prev;`
`}`

Aropan Dey
UG/104/BT CSE/2025/031

27. `void deleteNode (struct node* p)`
`{`
`struct node* temp;`
`temp = p->next;`
`p->next = temp->next;`
`free (temp);`
`}`

SUPPOSE we have: 40->20->30->NULL
 If we want to delete 20, make 20's next directly to 40.
 So, 40->30->40->20->...
 The code can be:
~~temp = p->next;~~

28. Initial list:

5->8->12->3->NULL

i) Insert 7 at the beginning:

7->5->8->12->3->NULL

ii) Delete 12:

7->5->8->3->NULL

iii) Insert 9 at the end:

7->5->8->3->9->NULL

Aropan Dey
UG/104/BT CSE/2025/031

Final list:-

7->5->8->3->9->NULL